

hw12

May 4, 2026

```
[1]: from keras.datasets import cifar10
      (x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 30s
0us/step

```
[2]: print('x_train shape:', x_train.shape)
```

x_train shape: (50000, 32, 32, 3)

```
[3]: print('y_train shape:', y_train.shape)
```

y_train shape: (50000, 1)

```
[4]: print(x_train[0])
```

```
[[[ 59  62  63]
   [ 43  46  45]
   [ 50  48  43]
   ...
   [158 132 108]
   [152 125 102]
   [148 124 103]]

  [[ 16  20  20]
   [  0   0   0]
   [ 18   8   0]
   ...
   [123  88  55]
   [119  83  50]
   [122  87  57]]

  [[ 25  24  21]
   [ 16   7   0]
   [ 49  27   8]
   ...
   [118  84  50]
   [120  84  50]
   [109  73  42]]]
```

...

```
[[208 170 96]
 [201 153 34]
 [198 161 26]
```

...

```
[160 133 70]
 [ 56  31  7]
 [ 53  34 20]]
```

```
[[180 139 96]
 [173 123 42]
 [186 144 30]
```

...

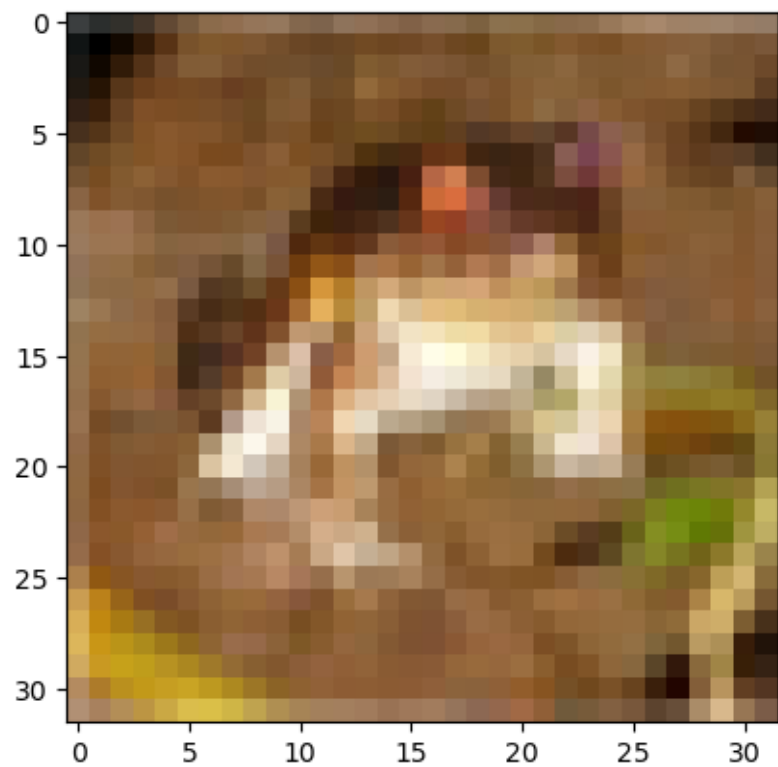
```
[184 148 94]
 [ 97  62 34]
 [ 83  53 34]]
```

```
[[177 144 116]
 [168 129 94]
 [179 142 87]
```

...

```
[216 184 140]
 [151 118 84]
 [123  92 72]]]
```

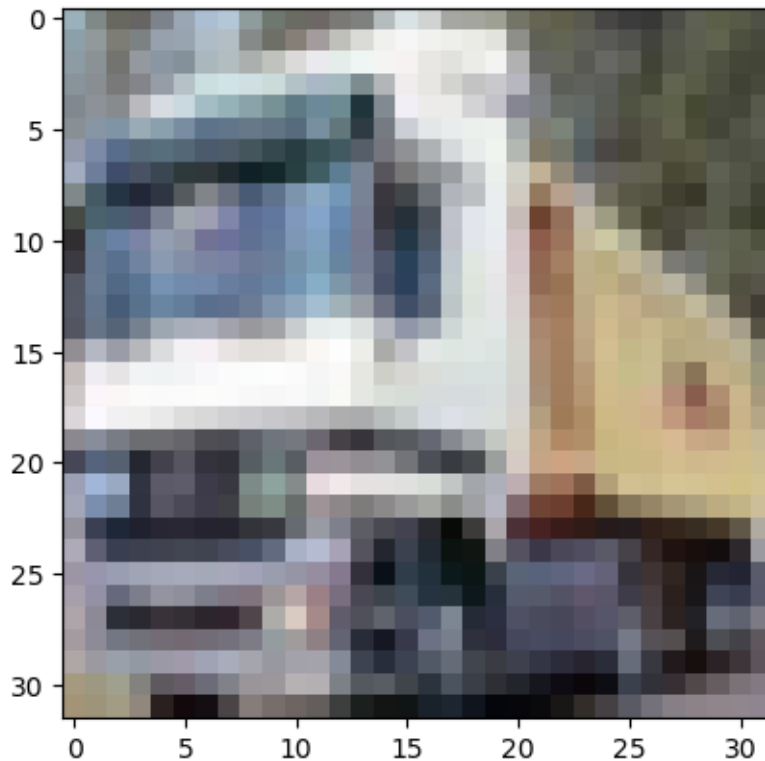
```
[5]: import matplotlib.pyplot as plt
      %matplotlib inline
      img = plt.imshow(x_train[0])
```



```
[6]: print('The label is:', y_train[0])
```

The label is: [6]

```
[7]: img = plt.imshow(x_train[1])
```



```
[8]: print('The label is:', y_train[1])
```

The label is: [9]

```
[9]: import keras
y_train_one_hot = keras.utils.to_categorical(y_train, 10)
y_test_one_hot = keras.utils.to_categorical(y_test, 10)
```

```
[10]: print('The one hot label is:', y_train_one_hot[1])
```

The one hot label is: [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]

```
[11]: x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train = x_train / 255
x_test = x_test / 255
```

```
[12]: x_train[0]
```

```
[12]: array([[0.23137255, 0.24313726, 0.24705882],
          [0.16862746, 0.18039216, 0.1764706 ],
          [0.19607843, 0.1882353 , 0.16862746],
          ...,
```

```

[0.61960787, 0.5176471 , 0.42352942],
[0.59607846, 0.49019608, 0.4          ],
[0.5803922 , 0.4862745 , 0.40392157]],

[[0.0627451 , 0.07843138, 0.07843138],
 [0.          , 0.          , 0.          ],
 [0.07058824, 0.03137255, 0.          ],
 ...,
 [0.48235294, 0.34509805, 0.21568628],
 [0.46666667, 0.3254902 , 0.19607843],
 [0.47843137, 0.34117648, 0.22352941]],

[[0.09803922, 0.09411765, 0.08235294],
 [0.0627451 , 0.02745098, 0.          ],
 [0.19215687, 0.10588235, 0.03137255],
 ...,
 [0.4627451 , 0.32941177, 0.19607843],
 [0.47058824, 0.32941177, 0.19607843],
 [0.42745098, 0.28627452, 0.16470589]],

...,

[[0.8156863 , 0.6666667 , 0.3764706 ],
 [0.7882353 , 0.6          , 0.13333334],
 [0.7764706 , 0.6313726 , 0.10196079],
 ...,
 [0.627451 , 0.52156866, 0.27450982],
 [0.21960784, 0.12156863, 0.02745098],
 [0.20784314, 0.13333334, 0.07843138]],

[[0.7058824 , 0.54509807, 0.3764706 ],
 [0.6784314 , 0.48235294, 0.16470589],
 [0.7294118 , 0.5647059 , 0.11764706],
 ...,
 [0.72156864, 0.5803922 , 0.36862746],
 [0.38039216, 0.24313726, 0.13333334],
 [0.3254902 , 0.20784314, 0.13333334]],

[[0.69411767, 0.5647059 , 0.45490196],
 [0.65882355, 0.5058824 , 0.36862746],
 [0.7019608 , 0.5568628 , 0.34117648],
 ...,
 [0.84705883, 0.72156864, 0.54901963],
 [0.5921569 , 0.4627451 , 0.32941177],
 [0.48235294, 0.36078432, 0.28235295]]], dtype=float32)

```

```
[13]: from keras.models import Sequential
      from keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
```

```
[14]: model = Sequential()
```

```
[15]: model.add(Conv2D(32, (3, 3), activation='relu', padding='same',
      ↪input_shape=(32,32,3)))
```

```
/opt/anaconda3/lib/python3.13/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
```

```
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[16]: model.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
```

```
[17]: model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
[18]: model.add(Dropout(0.25))
```

```
[19]: model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
      model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
      model.add(MaxPooling2D(pool_size=(2, 2)))
      model.add(Dropout(0.25))
```

```
[20]: model.add(Flatten())
```

```
[21]: model.add(Dense(512, activation='relu'))
```

```
[22]: model.add(Dropout(0.5))
```

```
[23]: model.add(Dense(10, activation='softmax'))
```

```
[24]: model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0

conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 512)	2,097,664
dropout_2 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 10)	5,130

Total params: 2,168,362 (8.27 MB)

Trainable params: 2,168,362 (8.27 MB)

Non-trainable params: 0 (0.00 B)

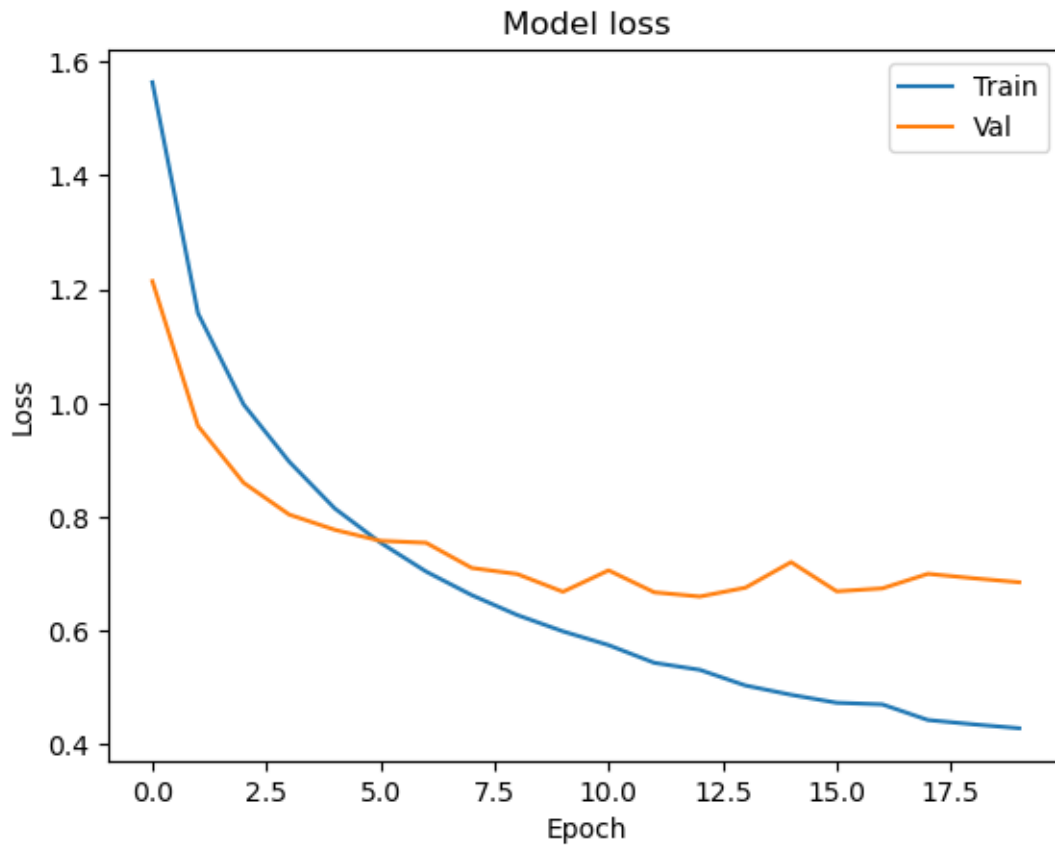
```
[25]: model.compile(loss='categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])
```

```
[26]: hist = model.fit(x_train, y_train_one_hot,
                       batch_size=32, epochs=20,
                       validation_split=0.2)
```

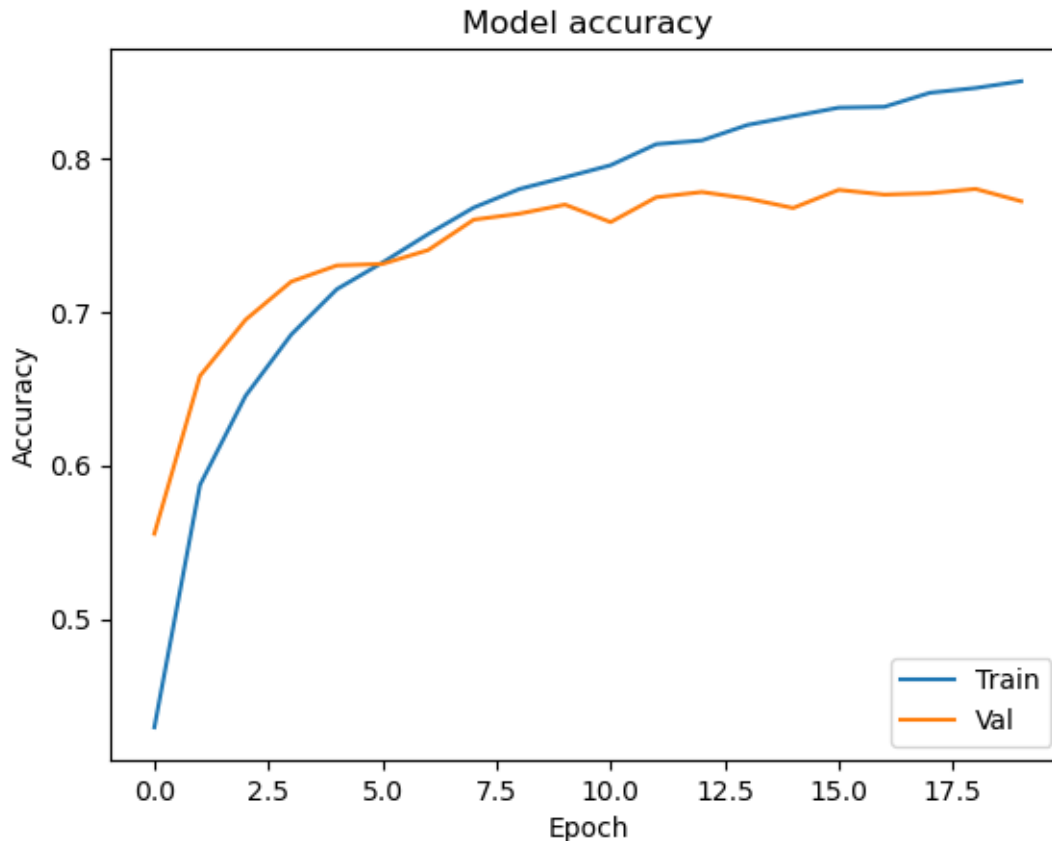
```
Epoch 1/20
1250/1250          42s 33ms/step -
accuracy: 0.4296 - loss: 1.5641 - val_accuracy: 0.5556 - val_loss: 1.2144
Epoch 2/20
1250/1250          40s 32ms/step -
accuracy: 0.5874 - loss: 1.1579 - val_accuracy: 0.6583 - val_loss: 0.9600
Epoch 3/20
1250/1250          41s 33ms/step -
accuracy: 0.6454 - loss: 0.9977 - val_accuracy: 0.6949 - val_loss: 0.8600
Epoch 4/20
1250/1250          41s 32ms/step -
accuracy: 0.6852 - loss: 0.8970 - val_accuracy: 0.7197 - val_loss: 0.8041
Epoch 5/20
1250/1250          40s 32ms/step -
```

accuracy: 0.7147 - loss: 0.8147 - val_accuracy: 0.7302 - val_loss: 0.7773
 Epoch 6/20
 1250/1250 41s 32ms/step -
 accuracy: 0.7321 - loss: 0.7548 - val_accuracy: 0.7312 - val_loss: 0.7580
 Epoch 7/20
 1250/1250 40s 32ms/step -
 accuracy: 0.7506 - loss: 0.7040 - val_accuracy: 0.7402 - val_loss: 0.7545
 Epoch 8/20
 1250/1250 42s 34ms/step -
 accuracy: 0.7679 - loss: 0.6625 - val_accuracy: 0.7600 - val_loss: 0.7103
 Epoch 9/20
 1250/1250 44s 36ms/step -
 accuracy: 0.7800 - loss: 0.6275 - val_accuracy: 0.7639 - val_loss: 0.6995
 Epoch 10/20
 1250/1250 43s 35ms/step -
 accuracy: 0.7876 - loss: 0.5988 - val_accuracy: 0.7698 - val_loss: 0.6684
 Epoch 11/20
 1250/1250 44s 35ms/step -
 accuracy: 0.7954 - loss: 0.5746 - val_accuracy: 0.7584 - val_loss: 0.7061
 Epoch 12/20
 1250/1250 42s 33ms/step -
 accuracy: 0.8091 - loss: 0.5436 - val_accuracy: 0.7746 - val_loss: 0.6675
 Epoch 13/20
 1250/1250 41s 33ms/step -
 accuracy: 0.8116 - loss: 0.5312 - val_accuracy: 0.7780 - val_loss: 0.6600
 Epoch 14/20
 1250/1250 41s 33ms/step -
 accuracy: 0.8217 - loss: 0.5037 - val_accuracy: 0.7738 - val_loss: 0.6756
 Epoch 15/20
 1250/1250 41s 32ms/step -
 accuracy: 0.8273 - loss: 0.4873 - val_accuracy: 0.7676 - val_loss: 0.7205
 Epoch 16/20
 1250/1250 41s 33ms/step -
 accuracy: 0.8329 - loss: 0.4733 - val_accuracy: 0.7794 - val_loss: 0.6692
 Epoch 17/20
 1250/1250 41s 33ms/step -
 accuracy: 0.8335 - loss: 0.4704 - val_accuracy: 0.7763 - val_loss: 0.6744
 Epoch 18/20
 1250/1250 40s 32ms/step -
 accuracy: 0.8426 - loss: 0.4430 - val_accuracy: 0.7772 - val_loss: 0.6999
 Epoch 19/20
 1250/1250 46s 37ms/step -
 accuracy: 0.8457 - loss: 0.4353 - val_accuracy: 0.7800 - val_loss: 0.6921
 Epoch 20/20
 1250/1250 45s 36ms/step -
 accuracy: 0.8501 - loss: 0.4285 - val_accuracy: 0.7720 - val_loss: 0.6851


```
[27]: plt.plot(hist.history['loss'])
plt.plot(hist.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Val'], loc='upper right')
plt.show()
```



```
[38]: plt.plot(hist.history['accuracy'])
plt.plot(hist.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Val'], loc='lower right')
plt.show()
```



```
[39]: model.evaluate(x_test, y_test_one_hot)[1]
```

```
313/313          3s 9ms/step -
accuracy: 0.7687 - loss: 0.6900
```

```
[39]: 0.7687000036239624
```

```
[40]: model.save('my_cifar10_model.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

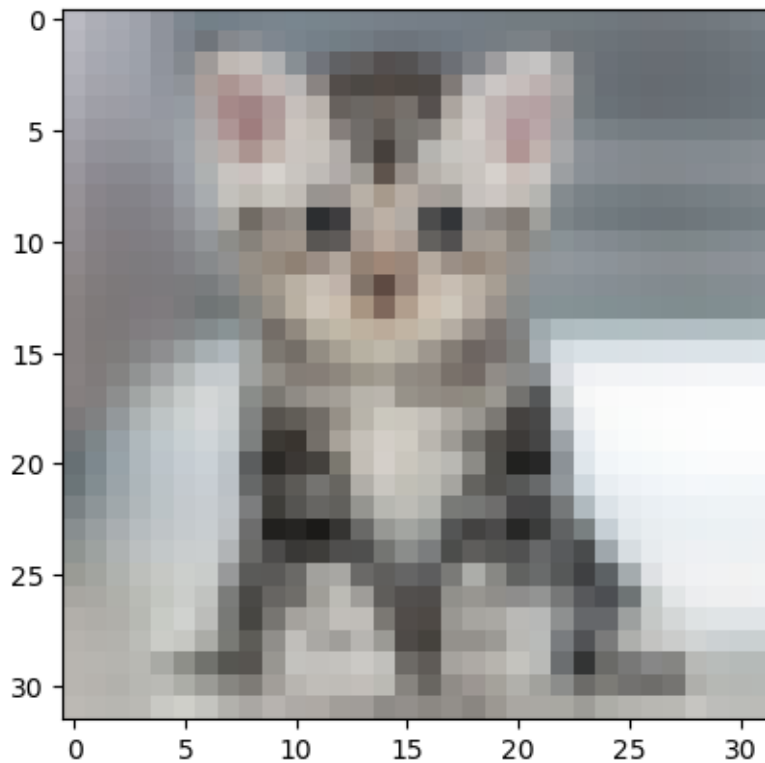
```
[41]: from keras.models import load_model
model = load_model('my_cifar10_model.h5')
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be
built. `model.compile_metrics` will be empty until you train or evaluate the
model.
```

```
[42]: my_image = plt.imread("cat.jpg")
```

```
[43]: from skimage.transform import resize
my_image_resized = resize(my_image, (32,32,3))
```

```
[44]: img = plt.imshow(my_image_resized)
```



```
[45]: import numpy as np
probabilities = model.predict(np.array( [my_image_resized,] ))
```

1/1 0s 56ms/step

```
[46]: probabilities
```

```
[46]: array([[1.2604673e-05, 1.3483002e-08, 2.8416788e-04, 2.3068628e-01,
            1.0220702e-01, 5.8047670e-01, 1.9893956e-05, 8.6312212e-02,
            8.9161153e-07, 3.0672251e-07]], dtype=float32)
```

```
[47]: number_to_class = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
                        ↪ 'frog', 'horse', 'ship', 'truck']
index = np.argsort(probabilities[0,:])
```

```
print("Most likely class:", number_to_class[index[9]], "-- Probability:",  
      ↪probabilities[0,index[9]])  
print("Second most likely class:", number_to_class[index[8]], "-- Probability:  
      ↪", probabilities[0,index[8]])  
print("Third most likely class:", number_to_class[index[7]], "-- Probability:",  
      ↪probabilities[0,index[7]])  
print("Fourth most likely class:", number_to_class[index[6]], "-- Probability:  
      ↪", probabilities[0,index[6]])  
print("Fifth most likely class:", number_to_class[index[5]], "-- Probability:",  
      ↪probabilities[0,index[5]])
```

Most likely class: dog -- Probability: 0.5804767
Second most likely class: cat -- Probability: 0.23068628
Third most likely class: deer -- Probability: 0.10220702
Fourth most likely class: horse -- Probability: 0.08631221
Fifth most likely class: bird -- Probability: 0.00028416788

[]: